

Beginner's Dart Cheat Sheet

Core Syntax, Data Structures, and Control Flow

Designed for Students & Developers

Covers Dart 3.x stable syntax

VARIABLES & TYPES

Variables store data values. Dart uses strong static type safety along with implicit type inference using the `var` keyword.

Hello World:

```
void main() {  
  print('Hello, World!');  
}
```

Variable Declarations:

```
var city = 'Delhi';      // Inferred as String  
String name = 'Anya';   // Explicit type  
final age = 21;          // Set once only  
const pi = 3.14159;     // Compile-time constant
```

STRING INTERPOLATION

Combine variables or expressions inside string literals easily using `$` signs.

```
String firstName = 'Nikola';  
String lastName = 'Tesla';  
// Combine variables directly  
String full = '$firstName $lastName';  
  
// Use curly braces for expressions  
print('Length: ${full.length}');
```

LISTS (ORDERED ARRAYS)

Lists store an ordered sequence of elements. Access elements using zero-based index arrays or iterate using standard loops.

Initialization & Access:

```
List<String> tech = ['Flutter', 'Dart', 'Web'];  
String first = tech[0]; // 'Flutter'  
String last = tech.last; // 'Web'
```

Modifying & Growing Lists:

```
var numbers = <int>[]; // Empty int list  
numbers.add(10);      // Appends 10  
numbers.addAll([20, 30]); // Appends list  
numbers.insert(1, 15); // Inserts at index 1
```

Collection For Syntax:

```
var base = [1, 2, 3];  
var squares = [for (var x in base) x * x];  
// squares container becomes: [1, 4, 9]
```

SETS & MAPS

Sets: Unordered collections of unique elements. Duplicates are rejected automatically.

```
var elements = {'Fluorine', 'Chlorine'};  
elements.add('Bromine');  
elements.add('Fluorine'); // Ignored safely
```

Maps: Unordered collection of key-value pairs where every key must be completely unique.

```
var alien = {'color': 'green', 'points': 5};  
print(alien['color']); // Output: green  
  
alien['x_position'] = 0; // Adds new pair  
  
alien.forEach((key, value) {  
  print('$key maps to $value');  
});
```

CONTROL FLOW & LOGIC

Conditional Tests:

```
if (age >= 18 && canVote) {  
  print('Access granted.');} else if (age < 18) {  
  print('Too young.');} else {  
  print('Access denied.');}
```

While Loops: Repeats code block while condition stays true.

```
int counter = 1;  
while (counter <= 5) {  
  print(counter);  
  counter++;  
}
```

Beginner's Dart Cheat Sheet

Functions, Object-Oriented Design, Exceptions, and Asynchronous Frameworks

Student Reference Material

Online resources: dart.dev

ADVANCED FUNCTIONS

Functions are isolated, structural blocks of code designed to solve a singular processing objective cleanly.

Standard Positional Function:

```
int addNumbers(int x, int y) {  
  return x + y;  
}
```

Named Parameters & Default Values:

Parameters enclosed inside curly braces are optional and referenceable by name during invocation loops.

```
void makePizza({String topping = 'bacon', int size = 12}) {  
  print('A $size-inch pizza with $topping.');
```

```
}  
  
void main() {  
  makePizza(); // Uses default values  
  makePizza(topping: 'pepperoni');  
}
```

Arrow Syntax Shorthand:

```
bool isEven(int n) => n % 2 == 0;
```

OBJECT-ORIENTED CODING

Classes act as structural object blueprints mapping individual variable attributes alongside function methods.

Declaring a Base Class:

```
class Dog {  
  String name;  
  
  // Idiomatic Constructor Shorthand  
  Dog(this.name);  
  
  void sit() {  
    print('$name is sitting.');  }  
}
```

Class Inheritance & Overrides:

```
class SARDog extends Dog {  
  String specialization;  
  
  // Forward fields up to Super class  
  SARDog(String name, this.specialization)  
    : super(name);  
  
  void search() {  
    print('$name is searching.');  }  
  
  @override  
  void sit() {  
    print('$name settles into a sit.');  }  
}
```

ASYNCHRONOUS PIPELINE

Dart executes sequentially on single isolation loops. Async functions yield Future frameworks representing deferred operations.

Async / Await Syntax:

```
Future<String> fetchMetadata() async {  
  return Future.delayed(  
    Duration(seconds: 2),  
    () => 'Data Payload Recieved'  
  );  
}  
  
void main() async {  
  print('Requesting data pipeline...');  
  String detail = await fetchMetadata();  
  print(detail);  
}
```

EXCEPTION HANDLING

Gracefully trap computational logic failures or explicit system crashes using structural try-catch blocks.

```
try {  
  int result = 5 ~/ 0; // Integer division  
  print(result);  
} on UnsupportedError {  
  print('Cannot divide by zero value.');} catch (error) {  
  print('Generic failure caught: $error');} finally {  
  print('This block always runs.');}
```